

nsx-analyzer

사용법

수집한 결과를 읽고 해석하기
어디서 사라졌는지를 찾는 도구

수집기는 모으기만 하고, 해석은 이 스크립트가 한다.

pcap 을 Wireshark 로 하나하나 열어 보는 대신, **5-tuple** 하나를 주면 모든 캡처 지점을 훑어 "어디까지 갔고 어디서 사라졌는지"를 말해 준다. 카운터·세션 테이블도 같이 읽어 드롭이 늘었는지, 세션이 언제 사라졌는지를 붙여 준다.

장비가 아니라 내 노트북에서 돌린다. 읽기만 하므로 몇 번을 돌려도 안전하다.

도구	nsx-analyzer v6.3
짹	nsx-collector 사용법 (수집 편)
작성일	2026-09-22

목차

Part 1	분석기가 하는 일	3
Part 2	결과 가져오기와 실행	5
Part 3	overview — 무엇이 들어 있나	7
Part 4	flow — 어디서 사라졌나 ★	8
Part 5	state — 카운터가 어떻게 움직였나	10
Part 6	session — 세션 · LB · DFW	11
Part 7	report — 한 파일로	13
Part 8	판독 치트시트	14

순서만 지키면 된다

overview	→	flow	→	state · session	→	report
무엇이		어디서		왜		보고서로
있나		사라졌나		그랬나		

Part 4(flow)가 이 도구의 핵심이다. 나머지는 거기서 나온 답을 뒷받침한다.

분석기가 하는 일

수집기와 역할이 나뉘어 있다

하는 일	스크립트	어디서
모으기	<code>nsx-collector.py</code>	조사 대상 장비 (Edge · ESXi)
해석하기	<code>nsx-analyzer.py</code>	아무 데서나 — 보통 내 노트북

두 스크립트는 **폴더 구조만 공유**하고 서로를 부르지 않는다. 수집기가 만든 `run-<장비>-<날짜>-<시각>/` 를 분석기가 읽는 방식이다.

왜 분리했나

- 장비에 부담을 주지 않는다 — 분석은 CPU 를 꽤 쓴다. 운영 Edge 에서 할 일이 아니다
- 몇 번이고 다시 볼 수 있다 — 같은 데이터를 조건만 바꿔 계속 돌린다
- 수집은 짧게, 분석은 길게 — 현장에서는 뜨는 데만 집중하고 해석은 나와서 한다

무엇을 읽어 주나

명령	답해 주는 질문
<code>overview</code>	이 수집에 무엇이 들어 있나 — 지점·기간·건수
<code>flow</code>	이 통신이 어디까지 갔고 어디서 사라졌나 ★
<code>state</code>	그 시간 동안 드롭·에러가 늘었나, HA 가 바뀌었나
<code>session</code>	세션이 있었나 없었나, LB 풀은 살아 있었나
<code>report</code>	위 전부를 파일 하나로

pcap 을 직접 해석한다

Wireshark 를 열지 않아도 된다. 그리고 두 가지를 알아서 처리한다.

처리	왜 중요한가
VLAN 태그 패킷	Edge 캡처는 절반쯤이 802.1Q 태그가 붙은 채 잡힌다. tcpdump 로 손수 볼 때 vlan 분기를 안 넣으면 조용히 절반만 보인다. 분석기는 양쪽을 다 센다
pcapng 형식	ESXi 의 pktcap-uw 는 pcapng 로 떨군다. 그대로 읽는다

결과 가져오기와 실행

장비에서 결과를 가져온다

```
scp -r root@<장비>:/var/dump/nsx-collect/run-<장비>-20260912-073308 .
```

ESXi 라면 `OUT` 에 지정한 데이터스토어 경로에서 가져온다.

```
scp -r root@<esxi>:/vmfs/volumes/<데이터스토어>/nsxc/run-* .
```

돌린다

```
python3 nsx-analyzer.py                # 메뉴
python3 nsx-analyzer.py --run ./run-... # 폴더를 직접 지정
python3 nsx-analyzer.py <동작>         # 메뉴 없이 그 동작만
python3 nsx-analyzer.py help
```

실행 폴더를 찾는 순서

순서	찾는 곳	
1	<code>--run <경로></code>	직접 준 것이 최우선
2	<code>/var/dump/nsx-collect</code>	Edge 위에서 바로 돌릴 때
3	<code>/tmp/nsx-collect</code>	ESXi 기본 경로
4	현재 폴더	가져온 폴더 옆에서 돌릴 때

여러 개가 있으면 **목록에서 고른다** — 메뉴의 `l` 또는 `runs` .

메뉴

```
+=====+
| NSX Analyzer 6.3                               |
| run: run-edge02-20260912-073308                |
+=====+
| 1 overview      what this run contains          |
| 2 flow          find a flow (5-tuple) and explain it |
| 3 state         counters, HA state, rising drops  |
| 4 session       firewall connections, LB, DFW    |
| 5 report        all of it, saved to a file        |
| r pick another run      9 help                    |
| l list runs            q quit                     |
+=====+
```

읽기만 한다

분석기는 **파일을 고치지 않는다**. `report` 로 보고서를 저장할 때만 파일을 만든다. 몇 번을 돌려도 원본은 그대로다. 마음껏 조건을 바꿔 가며 돌리면 된다.

overview

무엇이 들어 있나

항상 여기서 시작한다. 없는 데이터를 근거로 결론을 내지 않기 위해서다.

보여 주는 것

항목	내용
수집 당시 정보	케이스 번호 · 장비 · 실제로 사용된 필터식
파일 현황	pcap / state / session 별 파일 수와 크기
지점별 패킷 수	캡처 지점마다 몇 건이 잡혔고 시간 범위가 어떻게 되는지, 그 지점이 무엇을 보는 곳인지 설명까지
오류만 든 파일	명령이 거부되어 데이터 대신 에러 메시지만 들어 있는 파일을 따로 알려 준다

"오류만 든 파일" 경고를 반드시 보고 넘어간다

수집기가 그 NSX 버전에 없는 명령을 쓰면 파일은 만들어지는데 **안에는 에러 문구만** 들어 있다. 파일 목록만 보면 다 모인 것처럼 보이지만 실은 빈 껍데기다.

이걸 모르고 들어가면 없는 데이터를 근거로 "이상 없음"이라고 보고하게 된다. v6 에서 Edge 명령 4개를 고친 것도 이것 때문이다.

지점별 패킷 수가 0 이면 거기서 멈춘다

flow 를 돌리기 전에 **애초에 그 지점에 데이터가 있는지**부터 확인한다. 0 건이면 분석의 문제가 아니라 **수집의 문제**다 — Standby Edge 에서 떴거나, 필터가 안 맞았거나, 그 시간에 트래픽이 없었다.

flow

어디서 사라졌나

이 도구의 핵심. 5-tuple 하나로 모든 지점을 훑는다.

쓰는 법

```
python3 nsx-analyzer.py flow --src 10.1.1.5 --dst 10.1.1.50 --dport 8080 --proto tcp
python3 nsx-analyzer.py flow --dst <NAT_IP> --dport 1813 --proto udp
```

- **값은 전부 선택**이다. 아는 것만 주면 된다
- **응답 방향도 함께** 찾는다. 출발지·목적지를 뒤집어 줄 필요가 없다

정확히 안 맞으면 한 번 더 찾아 준다

5-tuple 로 못 찾으면 **host X and port Y** 방식으로 넓혀 다시 찾고 알려 준다. **LB SNAT** 처럼 **포트가 임의로 바뀌는** 경우에 필요하다 — 나갈 때 포트가 달라지니 정확한 5-tuple 이 성립하지 않는다.

알려 주는 것

구분	내용
지점별	건수 · 방향 · 바이트, 첫/마지막 시각, 관찰된 주소 조합 = NAT 이 일어난 지점
TCP	SYN / SYN-ACK / RST / FIN 개수, 핸드셰이크 성공 여부와 RTT, 재전송, 제로 윈도우, 누가 먼저 끊었는지
UDP	요청 · 응답 수, 응답 시간 중앙값 · 최대, 30초를 넘으면 방화벽 세션 만료 경고, RADIUS 메시지 종류
ICMP	unreachable 사유, MTU 문제
지점 비교	같은 vNIC 의 pre / post 를 비교해 통과 / 일부 드롭 / 전량 드롭 으로 구분
끝에	같은 것을 손으로 확인할 tcpdump 명령 (Edge 용 vlan 분기 포함)

"지점 비교"가 결론을 만든다

DFW pre 에는 있는데 post 에 없으면 **DFW 가 막은 것**이다. 추정이 아니라 증거다. 이 한 줄이 나오면 조사가 끝난다.

UDP 30초 경고

응답이 30초를 넘겨 왔다면 그 사이에 방화벽 세션이 만료됐을 수 있다. GFW 의 UDP 타이머가 30초이기 때문이다. 응답은 왔는데 클라이언트가 못 받은 사고가 이렇게 난다.

여러 장비의 결과를 한 번에 — `--all-runs`

조사 하나가 실행 폴더 여러 개가 되는 경우가 많다.

- VPC T1 과 LB T1 의 Active 가 서로 다른 Edge → Edge 마다 폴더 1개
- 백엔드 VM 이 여러 ESXi 호스트에 흩어져 있음 → 호스트마다 폴더 1개

폴더들을 한 곳에 모아 두고 이렇게 하면 전부 훑는다.

```
python3 nsx-analyzer.py flow --all-runs --run <모아둔폴더> \
  --dst <VIP> --dport 80 --proto tcp
```

실측 출력 (Edge 2대):

```
2 run directory(ies): run-edge01-..., run-edge02-...
RUN run-edge01-...   NOT FOUND here (0 packet(s) in the file)
RUN run-edge02-...   292 packet(s) of this flow out of 965 in the file
```

이게 진짜 무기다

Edge 와 ESXi 결과를 한 번에 놓고 같은 5-tuple 을 추적하면 "TO 에는 보이는데 LB T1 에는 없다", "pre 에는 있는데 post 에 없다" 같은 구간 판정이 바로 나온다.

state

카운터가 어떻게 움직였나

flow 가 찾은 현상의 뒷받침. "왜 그랬나"를 여기서 본다.

보여 주는 것

항목	내용
라우터 HA 상태	샘플마다 보여 주고, 중간에 바뀌면 <-- CHANGED 로 표시
Edge 인터페이스	fp-eth* · eth* 와 논리 라우터 인터페이스의 RX/TX 증감
ESXi 카운터	업링크 NIC, VM 스위치 포트 증감
오류 · 드롭	따로 모아서, 그 시간 동안 늘어난 것만 "먼저 볼 것"으로 표시

HA 가 바뀌었다면 거기서 답이 나온다

캡처 도중 Active / Standby 가 바뀌면 "갑자기 끊겼다"의 답이 여기 있다. 페일오버 순간 세션 상태가 넘어가지 못한 연결은 그대로 끊긴다.

<-- CHANGED 표시를 발견하면 그 시각과 장애 시각을 맞춰 본다.

카운터는 누적값이라 "증감"만 의미가 있다

한 번만 뜯 값은 아무 정보가 없다. 분석기는 샘플 간 차이를 계산해 보여 준다. 수집할 때 INTERVAL 로 여러 번 뜯 이유가 이것이다.

session

세션 · LB · DFW

"세션이 있었나 없었나"를 판정한다.

NSX Edge

항목	내용
연결 수 추이	인터페이스별로 샘플마다 — 9d7af199 073311=45, 073355=33
연결 테이블 분석	프로토콜 · 상태 분포, SYN 상태(half-open) 경고, 그리고 NAT 매핑을 그대로 10.0.0.1:61496 -> <VIP>:80 (as 10.0.0.1:80)
로드밸런서	샘플마다 상태와 가상 서버 · 풀 업 개수(다운이면 경고), L4/L7 현재 · 최대 · 누적 세션, 초당 세션 수
풀 멤버	멤버별 IP · 포트 · 상태와 헬스 모니터 상태

ESXi

항목	내용
DFW pass / drop	패킷 기준 카운터와 그 시간 동안의 증감 → 드롭이 늘면 경고하고 왜 떨어졌는지까지 (icmp error , state-mismatch , seqno outside window 등)
DFW 세션 표	건수 추이, 적용된 규칙 수와 drop/reject 규칙 수

드롭 사유까지 보여 준다

DFW 가 막았다는 것만이 아니라 어떤 종류의 드롭인지를 구분해 준다. state-mismatch 나 seqno outside window 가 늘었다면 정책 문제가 아니라 세션 상태 문제다 — 정책을 고쳐도 해결되지 않는다.

가장 최근 샘플을 "전부" 읽는다

연결 테이블과 DFW 세션 표는 인터페이스 · vNIC 마다 최근 샘플을 전부 읽는다. 파일 하나만 보면 한쪽만 보인다 — 실측으로 잡은 문제다.

특정 플로우만 보기

5-tuple 을 같이 주면 **연결 테이블**과 **DFW 세션** 표에서 그 플로우만 찾아 준다.

```
python3 nsx-analyzer.py session --dst 10.0.0.1 --dport 80 --proto tcp
```

report 한 파일로

쓰는 법

```
python3 nsx-analyzer.py report 분석결과.txt
python3 nsx-analyzer.py report 분석결과.txt --dst 10.1.1.50 --dport 8080 --proto tcp
```

overview + state + session 을 한 파일에 담는다. 5-tuple 을 주면 flow 까지 들어간다.

메뉴에서 5 를 누르면 실행 폴더 안에 99-analysis-<시각>.txt 로 저장된다.

고객 회신 · 케이스 첨부에 그대로 쓴다

손으로 정리할 필요 없이 근거가 붙은 한 파일이 나온다. 본사 케이스를 열 때도 이 파일을 첨부하면 된다.

정리하면 이 순서다

1. scp 로 결과를 가져온다
2. overview # 데이터가 제대로 있는지부터
3. flow # 어디서 사라졌나 ← 핵심
4. state # 드롭·에러가 늘었나, HA 가 바뀌었나
5. session # 세션이 있었나 없었나
6. report # 한 파일로 저장

판독 치트시트

출력에서 이게 보이면 이렇게 읽는다.

flow 에서

이게 보이면	이렇게 읽는다
NOT FOUND here (0 packet(s))	그 지점에는 이 흐름이 안 지나갔다. 앞 지점과 비교해 구간을 좁힌다
pre 에 있고 post 에 없음	DFW 가 막았다. 조사 끝
SYN 은 있는데 SYN-ACK 가 없음	서버까지 안 갔거나, 응답이 돌아오는 길에서 사라졌다
재전송이 많음	중간에서 유실되고 있다. 카운터(Part 5)로 넘어간다
주소 조합이 지점마다 다름	그 사이에서 NAT 이 일어났다. 정상일 수도, 예상 밖일 수도
UDP 응답 30초 초과 경고	방화벽 세션이 만료됐을 수 있다

state 에서

이게 보이면	이렇게 읽는다
<-- CHANGED (HA 상태)	페일오버가 있었다. 장애 시각과 맞춰 본다
드롭 카운터가 "먼저 볼 것"에	그 구간에서 실제로 버려지고 있다
RX 는 느는데 TX 가 안 늡	들어오기만 하고 못 나간다. 다음 흐름 본다

session 에서

이게 보이면	이렇게 읽는다
연결 수가 급감	세션이 대량으로 사라진 시점이 있다
SYN 상태(half-open) 경고	핸드셰이크가 안 끝나는 연결이 쌓인다
LB 풀 멤버 DOWN	백엔드 문제 . 네트워크를 파기 전에 VM 전원부터 확인
DFW <code>state-mismatch</code> 증가	정책이 아니라 세션 상태 문제 . 정책을 고쳐도 안 낫는다
NAT 매핑 (<code>as ...</code>)	그 연결의 원래 주소 . LB 경로 추적의 핵심

기억할 것 세 가지

- 1. **overview** 없이 시작하지 않는다 — 없는 데이터로 결론 내는 것이 최악이다
- 2. **지점 비교가 결론을 만든다** — 한 지점만 보면 아무것도 확정되지 않는다
- 3. **장비가 여러 대면 --all-runs** — 따로 보면 구간 판정이 안 된다